Critical Review: “What Makes a Great Software Engineer?”

The article *What Makes A Great Software Engineer?*, by Paul Luo Li, Amy J. Ko, and Jiamin Zhu, discusses an important question in the field of software engineering: what characteristics make someone a great software engineer? The authors argue that the concept of a “great engineer” is often described using vague expressions, such as being a good communicator or a good team player. Therefore, their main objective was to provide a more specific and detailed understanding of software engineering expertise. To investigate this topic, the researchers conducted 59 semi-structured interviews with experienced engineers from 13 different divisions at Microsoft. From these interviews, they identified 53 attributes associated with great software engineers.

One of the most interesting aspects of the article is that it challenges the idea that being a great software engineer is mainly about technical knowledge or programming speed. The researchers found that great engineers combine technical abilities with personal characteristics, decision-making skills, teamwork, and the ability to produce high-quality software. Some of the characteristics identified include being curious, passionate, open-minded, systematic, productive, perseverant, adaptable, self-aware, and focused. The study also emphasizes the importance of understanding people and organizations, communicating effectively, creating shared success, mentoring colleagues, and building trust within a team.

In my opinion, one of the strongest points of the article is its broader definition of software engineering. I agree with the authors that programming ability alone is not enough to make someone a great professional. Software development is usually a collaborative activity, and engineers need to communicate with teammates, managers, customers, and other stakeholders. The article’s discussion of communication is particularly relevant because it explains that effective communication is not simply about speaking clearly, but about creating a shared understanding of a problem. This is important because technical professionals frequently need to explain complex ideas to people with different levels of technical knowledge.

Another particularly valuable point is the importance given to continuous learning. The authors found that experienced engineers did not consider one specific technical skill to be essential for everyone. Instead, they emphasized the ability to learn new skills and continuously improve because technology and software development practices are constantly changing. I strongly agree with this conclusion. A programming language, framework, or tool can become outdated, but the ability to learn and adapt remains useful throughout a professional career. For students and people beginning their careers in technology, this is an especially relevant message.

The article also has important strengths from a methodological perspective. The researchers did not simply provide their personal opinions about what makes an engineer successful. They interviewed experienced professionals and analyzed more than 60 hours of interviews and 388,000 words of transcripts using a grounded theory approach. They also involved another senior software engineer in analyzing approximately one-third of the interviews to help validate the interpretations. This makes the study more systematic and provides a stronger foundation for its conclusions.

However, the study also has significant limitations. The most important one is that all participants worked at Microsoft and were located in Seattle. The researchers themselves acknowledge that the sample was relatively small and that only three of the 59 participants were women. They also recognize that the findings may not apply equally to other cultures, companies, industries, or smaller organizations such as startups. In my view, this is an important limitation because Microsoft is a very large technology company with its own culture, processes, tools, and expectations. What makes an

engineer successful at Microsoft may not necessarily be exactly the same in a small startup, a government organization, or a company whose main business is not software.

Another limitation is that many of the identified attributes are subjective. Characteristics such as being passionate, personable, prideful, or “great” at handling complexity can be difficult to measure objectively. The authors recognize this problem and suggest that future research should develop ways to measure these attributes and investigate their relationship with team and project outcomes. This is important because identifying characteristics is only the first step; researchers would also need to determine which characteristics have the greatest impact and whether they can actually be developed through education and professional experience.

Overall, I consider the article valuable because it presents a more realistic and complete view of what it means to be a great software engineer. Its main strength is showing that software engineering is not only a technical activity, but also a social and decision-making process. The article demonstrates that technical knowledge, communication, adaptability, continuous learning, teamwork, and good judgment all contribute to professional excellence. At the same time, its conclusions should not be treated as a universal definition of a great software engineer because the research was conducted within a single company and cultural context.

In conclusion, the article provides a useful framework for understanding software engineering expertise and is particularly relevant for students and professionals who want to improve their careers. I agree with the authors’ central argument that being a great software engineer is not simply about writing code quickly or knowing many technologies. In my opinion, a great engineer is someone who can continuously learn, make good decisions, understand the impact of their work, collaborate with others, and adapt to new situations. Despite its methodological limitations, the article successfully encourages readers to think beyond technical skills and recognize the human and organizational dimensions of software engineering.